

Lecture 8: Switch Statements, Functions, and Modular Programming

1. The switch Statement Paradigm

When evaluating a single variable against numerous specific, discrete values, chaining endless `else if` statements becomes computationally inefficient and difficult to read. The `switch` statement acts as a cleaner, highly optimized routing mechanism.

- **Routing Mechanism:** The `switch` evaluates an expression once and strictly jumps to the `case` block that matches the evaluated value.
- **The `break` Keyword:** In C++, if a `case` block is executed and does not encounter a `break` statement, the program will suffer from "fall-through," executing every subsequent case below it unconditionally. `break` forces the execution to exit the `switch` block entirely.
- **The default Case:** Acts as the universal fallback. If absolutely no case matches the expression, the `default` block executes (analogous to the final `else` in an `if-else` chain).
- **Infinite Loop Escape:** If a `switch` is nested inside an infinite loop (e.g., `while(1)`), using `break` will only break the `switch`, not the loop. To terminate the entire program from within a nested `switch`, you must invoke the `exit(0)` system function.

Mini-Project: Command-Line Calculator

A standard demonstration of the `switch` statement is building a basic mathematical router based on character input.

C++

```
#include<iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter the value of a: " << endl;
    cin >> a;
    cout << "Enter the value of b: " << endl;
    cin >> b;

    char op;
    cout << "Enter the Operation (+, -, *, /, %): " << endl;
    cin >> op;

    switch(op) {
        case '+': cout << (a + b) << endl; break;
```

```

        case '-': cout << (a - b) << endl; break;
        case '*': cout << (a * b) << endl; break;
        case '/': cout << (a / b) << endl; break;
        case '%': cout << (a % b) << endl; break;
        default: cout << "Please enter a valid Operation" << endl;
    }
    return 0;
}

```

2. Functions & Pass-by-Value Mechanics

As programs scale, writing massive, monolithic code inside `main()` becomes unmaintainable. Functions allow you to compartmentalize logic into reusable blocks.

- **Function Signature:** Dictates the rules of engagement. It includes the return type (what the function gives back), the Function Name, and the parameters (what the function requires to operate).
- **Void Type:** If a function strictly performs an action (like printing to the console) and does not mathematically compute a value to hand back to the program, its return type is `void`.
- **Pass-by-Value (CRITICAL):** When you pass a variable into a C++ function, you are **not** passing the original variable. The compiler allocates new memory and passes a *carbon copy* of the value. Any modifications made to the variable inside the function are completely destroyed when the function ends. The original variable in `main()` remains untouched.

Problem-Solving Deconstruction

Problem 1: Calculate Power (a^b)

1. **Problem Statement & Constraints:** Given a base integer a and an exponent integer b , compute the mathematical value of a^b using a modular function.
2. **Core Intuition:** Instead of rewriting `for` loops every time we need to calculate an exponent, we abstract the logic. The algorithm requires setting a running product to 1 and multiplying it by the base a exactly b times.
3. **Algorithmic Steps:**
 1. Declare function `int power(int num1, int num2)`.
 2. Initialize `ans = 1`.
 3. Open a `for` loop executing `num2` times (from $i=1$ up to `num2`).
 4. Update the accumulator: `ans = ans * num1`.
 5. `return ans` to the caller.
4. **Dry Run:**

Input: num1 = 2 , num2 = 3

1. ans = 1 .
2. i = 1: ans = 1 * 2 = 2 .
3. i = 2: ans = 2 * 2 = 4 .
4. i = 3: ans = 4 * 2 = 8 .
5. Loop ends. Returns 8.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(B)$	The loop scales linearly with the size of the exponent B .
Space Complexity	$O(1)$	Memory usage is fixed to scalar variables.

C++

```
#include<iostream>
using namespace std;

int power(int num1, int num2) {
    int ans = 1;
    for(int i = 1; i <= num2; i++) {
        ans = ans * num1;
    }
    return ans;
}

int main() {
    int c, d;
    cin >> c >> d;
    int answer = power(c, d);
    cout << "Answer is " << answer << endl;
    return 0;
}
```

Problem 2: Bitwise Even/Odd Checker

1. **Problem Statement & Constraints:** Write a boolean function to determine if a given integer is even or odd, prioritizing computational speed over standard modulo math.
2. **Core Intuition:** At the machine level, the binary representation of every odd number ends in a 1, and every even number ends in a 0. Applying a bitwise AND with a mask of 1 ($a \& 1$) will strictly isolate this Least Significant Bit (LSB).

3. Algorithmic Steps:

1. Declare function `bool isEven(int a) .`
2. Evaluate condition: If `a & 1` is True (evaluates to 1), the number is odd. Return 0 (False).
3. Else, the number is even. Return 1 (True).

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(1)$	Bitwise AND evaluates at the hardware level instantly.
Space Complexity	$O(1)$	No extra memory allocation required.

C++

```
#include<iostream>
using namespace std;

// Returns 1 if Even, 0 if Odd
bool isEven(int a) {
    if(a & 1) {
        return 0; // It's Odd
    } else {
        return 1; // It's Even
    }
}

int main() {
    int num;
    cin >> num;
    if(isEven(num)) {
        cout << "Number is Even" << endl;
    } else {
        cout << "Number is Odd" << endl;
    }
    return 0;
}
```

Problem 3: Combinatorics - Calculate nCr

1. Problem Statement & Constraints: Write a program to compute the mathematical combination formula: $nCr = \frac{n!}{r! \times (n-r)!}$.

2. Core Intuition: The formula requires calculating a factorial three separate times. Writing the factorial loop three times inside `main()` violates the DRY (Don't Repeat Yourself) principle. We must create a

standalone `factorial()` helper function and invoke it dynamically from an `nCr()` function.

3. Algorithmic Steps:

1. Define `int factorial(int n)` which loops from 1 to N , multiplying a running product `fact`.
2. Define `int nCr(int n, int r)`.
3. Compute the numerator: `num = factorial(n)`.
4. Compute the denominator: `denom = factorial(r) * factorial(n - r)`.
5. Return the integer division: `num / denom`.

4. Dry Run:

Input: $n = 8, r = 0$

1. `num = factorial(8)`.
2. `denom = factorial(0) * factorial(8 - 0)  $\rightarrow 1 \times 8!$` .
3. Result: $8! / (1 \times 8!) = 1$. Returns 1.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	The factorial function runs linearly up to N . Invoking it three times yields $O(3N)$, which reduces to $O(N)$.
Space Complexity	$O(1)$	Uses a fixed number of integer variables.

C++

```
#include<iostream>
using namespace std;

int factorial(int n) {
    int fact = 1;
    for(int i = 1; i <= n; i++) {
        fact = fact * i;
    }
    return fact;
}

int nCr(int n, int r) {
    int num = factorial(n);
    int denom = factorial(r) * factorial(n - r);
    return num / denom;
}

int main() {
```

```

int n, r;
cin >> n >> r;
cout << "Answer is " << nCr(n, r) << endl;
return 0;
}

```

Problem 4: Modular Prime Number Checker

1. Problem Statement & Constraints: Encapsulate prime checking logic into a modular boolean function returning True if prime, False if not.

2. Core Intuition: A prime number N is only divisible by 1 and N . If we check every integer from 2 up to $N-1$ and find any divisor that yields a modulo of 0, we can immediately abort the function and return False.

3. Algorithmic Steps:

1. Define `bool isPrime(int n)`.
2. Run a loop with iterator `i` from 2 up to $N-1$.
3. Check `if (n % i == 0)`. If True, we found a perfect divisor. Return 0 instantly (aborts function).
4. If the loop completely finishes without returning 0, no divisors exist. Return 1.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N)$	In the worst-case scenario (when the number is actually prime), it checks all $N-2$ factors.
Space Complexity	$O(1)$	Zero extra data structures utilized.

C++

```

#include<iostream>
using namespace std;

// 0 -> Not a Prime no.
// 1 -> Prime no.
bool isPrime(int n) {
    for(int i = 2; i < n; i++) {
        if(n % i == 0) {
            // Divide hogya hai, not a prime no.
            return 0;
        }
    }
    return 1;
}

```

```
int main() {  
    int n;  
    cin >> n;  
    if(isPrime(n)) {  
        cout << "is a Prime number" << endl;  
    } else {  
        cout << "Not a prime number" << endl;  
    }  
    return 0;  
}
```